

Lab 3, Hypothesis Testing Through Simulation

Type I Error, Type II Error, and Power Across 10,000 Samples

2026-09-29

Contents

Introduction	1
Building the population	2
Estimating the slope and its standard error in every sample	2
Testing a true null hypothesis	4
Testing a false null hypothesis	4
Seeing the two errors	5
Changing the significance level	6
Summary and questions to think about	8

Introduction

In Lab 2 we built a population where we knew the true relationship between a mother's education and her baby's birthweight, and we watched OLS estimates vary from one sample to the next. This lab picks up where that one ended. Knowing that estimates vary, how should a researcher decide whether a hypothesis about the slope is believable? The answer is a hypothesis test, and in this lab we study how well that test performs.

The setup is the same as before. We create a population of one million mothers and babies with a known slope, draw 10,000 samples of 10,000 observations each, and estimate the regression in every sample. This time we also compute the standard error of the slope and use it to test hypotheses whose truth we already know. Because we know the answer, we can count exactly how often the test gets it right and how often it gets it wrong.

By the end of this lab you should be able to do the following.

1. Build a population from a known data generating process and draw repeated samples from it.
2. Compute the OLS slope and its standard error in every sample and store them in a data frame.
3. Test a true null hypothesis in every sample and measure the Type I error rate.
4. Test a false null hypothesis in every sample and measure the Type II error rate and the power of the test.
5. Explain how changing the significance level trades one kind of error for the other.

```
library(ggplot2)
```

We fix the random number seed so that everyone gets the same numbers.

```
set.seed(345)
```

Building the population

The population has one million mother and child pairs. The variable `mothereduc` is the mother's years of education and `birthweight` is the baby's weight in ounces. The true relationship is given by the population regression function

$$\text{birthweight}_i = \beta_0 + \beta_1 \text{mothereduc}_i + u_i$$

where $\beta_0 = 104$, $\beta_1 = 1.5$, and u_i is a normally distributed error with mean zero and standard deviation 20 ounces.

Mother's education is drawn from a normal distribution with mean 12 and standard deviation 3, restricted to the range 0 to 23 years. We enforce the restriction by redrawing any value that falls outside the range, which keeps a proper normal shape without piling observations at the boundaries.

```
draw_truncated_normal <- function(n, mean, sd, lower, upper) {
  values <- numeric(0)
  while (length(values) < n) {
    remaining <- n - length(values)
    candidates <- rnorm(remaining * 1.2, mean = mean, sd = sd)
    candidates <- candidates[candidates >= lower & candidates <= upper]
    values <- c(values, candidates)
  }
  values[1:n]
}

N <- 1000000

beta0_true <- 104
beta1_true <- 1.5
sigma_error <- 20

mothereduc <- draw_truncated_normal(n = N, mean = 12, sd = 3, lower = 0, upper = 23)
error <- rnorm(N, mean = 0, sd = sigma_error)
birthweight <- beta0_true + beta1_true * mothereduc + error

population <- data.frame(mothereduc, birthweight)
summary(population)
```

```
##      mothereduc      birthweight
## Min.   : 0.002283  Min.   : 22.56
## 1st Qu.: 9.977072  1st Qu.:108.20
## Median :12.001261  Median :122.05
## Mean   :11.999902  Mean   :122.03
## 3rd Qu.:14.025743  3rd Qu.:135.84
## Max.   :22.994351  Max.   :219.11
```

Estimating the slope and its standard error in every sample

We now draw 10,000 independent samples of 10,000 observations each. In every sample we estimate the regression and record the slope, $\hat{\beta}_1$, along with its standard error. The standard error measures how much the slope estimate would vary from sample to sample, and it is the yardstick we need to judge whether an estimate is far from a hypothesized value.

For a simple regression the standard error of the slope is

$$se(\hat{\beta}_1) = \sqrt{\frac{\hat{\sigma}^2}{\sum_{i=1}^n (x_i - \bar{x})^2}} \quad \text{where} \quad \hat{\sigma}^2 = \frac{\sum_{i=1}^n \hat{u}_i^2}{n-2}$$

To keep the loop fast we use `.lm.fit()`, a bare bones version of `lm()` that returns only the coefficients and residuals. That is all we need, since we build the standard error by hand from the formula above. The results go into a data frame with one row per sample.

```
set.seed(345)

n_sims <- 10000
n_sample <- 10000

results <- data.frame(
  beta1_hat = numeric(n_sims),
  se_beta1 = numeric(n_sims)
)

for (i in 1:n_sims) {
  idx <- sample.int(N, n_sample)
  x_i <- mothereduc[idx]
  y_i <- birthweight[idx]

  fit_i <- .lm.fit(cbind(1, x_i), y_i)

  sigma2_i <- sum(fit_i$residuals^2) / (n_sample - 2)
  sxx_i <- sum((x_i - mean(x_i))^2)

  results$beta1_hat[i] <- fit_i$coefficients[2]
  results$se_beta1[i] <- sqrt(sigma2_i / sxx_i)
}

head(results)

##   beta1_hat   se_beta1
## 1  1.520950  0.06624239
## 2  1.534972  0.06663916
## 3  1.379524  0.06638449
## 4  1.469285  0.06686636
## 5  1.525199  0.06717287
## 6  1.492125  0.06622816
```

As a check, the average slope estimate should sit close to the true value of 1.5, and the average standard error should sit close to the actual spread of the slope estimates across samples.

```
c(
  mean_beta1_hat = mean(results$beta1_hat),
  mean_se_beta1 = mean(results$se_beta1),
  sd_of_beta1_hat = sd(results$beta1_hat)
)

##   mean_beta1_hat   mean_se_beta1 sd_of_beta1_hat
##           1.50446535           0.06674497           0.06659049
```

Testing a true null hypothesis

Suppose a researcher in each of the 10,000 samples tests

$$H_0: \beta_1 = 1.5 \quad \text{against} \quad H_1: \beta_1 \neq 1.5$$

using the t statistic

$$t = \frac{\hat{\beta}_1 - 1.5}{se(\hat{\beta}_1)}$$

and a 5 percent significance level. We reject H_0 when the absolute value of t exceeds the critical value from a t distribution with $n - 2$ degrees of freedom. Since we built the population with $\beta_1 = 1.5$, this null is true, and the correct decision in every sample is to fail to reject.

We store the decision in a vector where 1 means we fail to reject H_0 and 0 means we reject it.

```
alpha <- 0.05
crit <- qt(1 - alpha / 2, df = n_sample - 2)

results$t_true_null <- (results$beta1_hat - 1.5) / results$se_beta1
results$fail_to_reject_true <- as.numeric(abs(results$t_true_null) < crit)

correct_rate <- mean(results$fail_to_reject_true)
type1_error <- 1 - correct_rate

c(share_correct = correct_rate, type1_error = type1_error)

## share_correct    type1_error
##           0.9509           0.0491
```

The mean of a vector of zeros and ones is the share of ones, so `correct_rate` is the fraction of samples in which the test reached the right conclusion. The remaining fraction, `type1_error`, is the share of samples in which the test rejected a null that was true. This is called a Type I error. It should land close to 5 percent, because the significance level is by definition the probability of rejecting a true null. Even a perfectly specified test is wrong about 1 time in 20, and the small gap between the simulated rate and 5 percent is Monte Carlo noise.

Testing a false null hypothesis

We now test a null that is false. Suppose the researchers instead test

$$H_0: \beta_1 = 1.3 \quad \text{against} \quad H_1: \beta_1 \neq 1.3$$

The true slope is 1.5, so the correct decision in every sample is to reject. We build the same 0 and 1 vector, but a 1 is now a mistake.

```
results$t_false_null <- (results$beta1_hat - 1.3) / results$se_beta1
results$fail_to_reject_false <- as.numeric(abs(results$t_false_null) < crit)

type2_error <- mean(results$fail_to_reject_false)
power <- 1 - type2_error

c(type2_error = type2_error, power = power)
```

```
## type2_error      power
##      0.1347      0.8653
```

Here the mean of the vector is the share of samples in which the test failed to reject a false null. This is called a Type II error. One minus that share is the power of the test, which is the probability of correctly rejecting a false null.

Power depends on how far the truth sits from the null relative to the sampling noise. The true gap is 0.2 and the standard error of the slope is about 0.067, so the typical t statistic is around 3.1. That is beyond the critical value of 1.96 in most samples but not in all of them, which is why the Type II error rate is well above zero. A null further from the truth, a larger sample, or a smaller error variance would all raise power.

Seeing the two errors

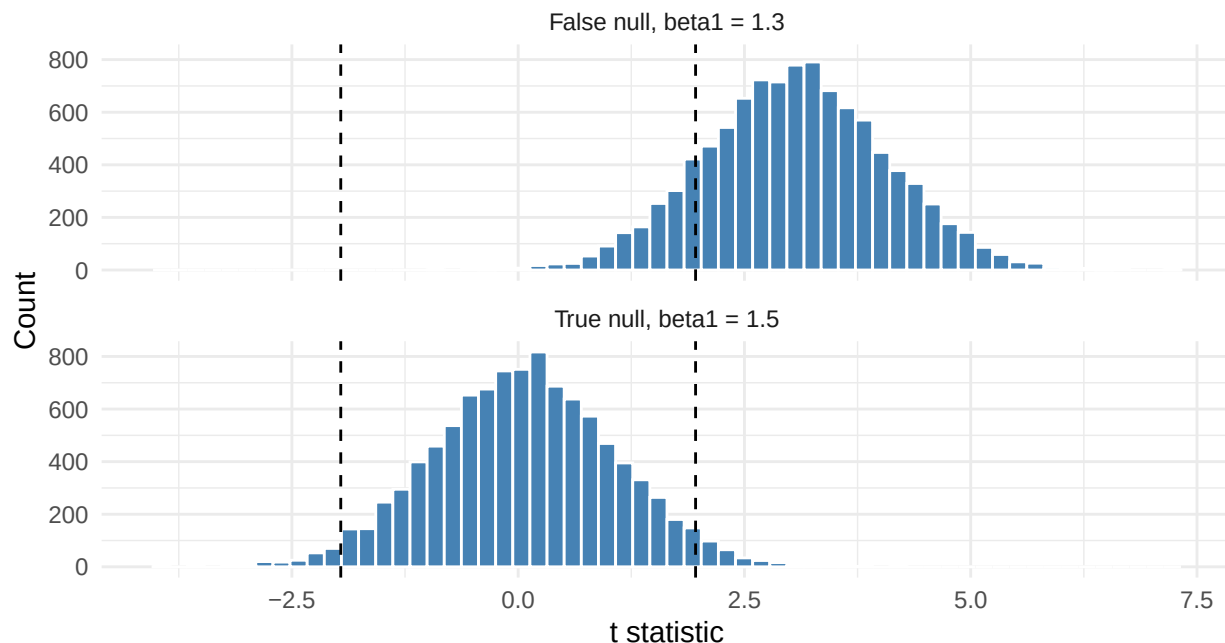
The two histograms below show the t statistic in each case. When the null is true the statistics center on zero and only the tails beyond the dashed lines lead to rejection, which is the Type I error. When the null is false the statistics are shifted to the right, and only the part of the distribution that falls between the dashed lines leads to a mistaken failure to reject, which is the Type II error.

```
plot_data <- data.frame(
  t_stat = c(results$t_true_null, results$t_false_null),
  null = rep(c("True null, beta1 = 1.5", "False null, beta1 = 1.3"), each = n_sims)
)

ggplot(plot_data, aes(x = t_stat)) +
  geom_histogram(bins = 60, fill = "steelblue", color = "white") +
  geom_vline(xintercept = c(-crit, crit), linetype = "dashed") +
  facet_wrap(~ null, ncol = 1) +
  labs(
    title = "Distribution of the t statistic under a true and a false null",
    subtitle = "Dashed lines mark the 5 percent critical values",
    x = "t statistic",
    y = "Count"
  ) +
  theme_minimal()
```

Distribution of the t statistic under a true and a false null

Dashed lines mark the 5 percent critical values



Changing the significance level

Everything so far used a 5 percent significance level. That choice is a convention, not a law of nature, and it directly sets how demanding the test is. A lower significance level raises the critical value, so the t statistic has to be further from zero before we reject. We now rerun both tests at 10 percent, 5 percent, and 1 percent, using the same 10,000 samples so that any difference in the results comes only from the significance level.

```
error_rates <- function(alpha) {
  crit_a <- qt(1 - alpha / 2, df = n_sample - 2)
  type1 <- mean(abs(results$t_true_null) >= crit_a)
  type2 <- mean(abs(results$t_false_null) < crit_a)
  data.frame(
    significance_level = alpha,
    critical_value = crit_a,
    type1_error = type1,
    type2_error = type2,
    power = 1 - type2
  )
}

alpha_table <- do.call(rbind, lapply(c(0.10, 0.05, 0.01), error_rates))
knitr::kable(alpha_table, digits = 4)
```

significance_level	critical_value	type1_error	type2_error	power
0.10	1.6450	0.1002	0.0782	0.9218
0.05	1.9602	0.0491	0.1347	0.8653
0.01	2.5763	0.0103	0.3121	0.6879

The table shows a clear trade off. Moving from 10 percent to 1 percent, the critical value rises from 1.65 to

2.58. The Type I error rate falls from about 10 percent to about 1 percent, so we wrongly reject a true null far less often. But the Type II error rate climbs from about 7.8 percent to about 31.2 percent, so we miss the false null much more often, and power drops by the same amount.

The reason is that the two errors pull on the same critical value from opposite sides. A wider acceptance region protects us against rejecting a true null, but it also swallows more of the t statistics that come from a false null. No choice of significance level removes both errors at once. The only ways to reduce both together are to gather more data or to face a less noisy error term, since either one shrinks the standard error and pulls the two distributions further apart.

The figure below traces the same trade off over a fine grid of significance levels. The Type I error rate rises along the 45 degree line, since it equals the significance level, while the Type II error rate falls.

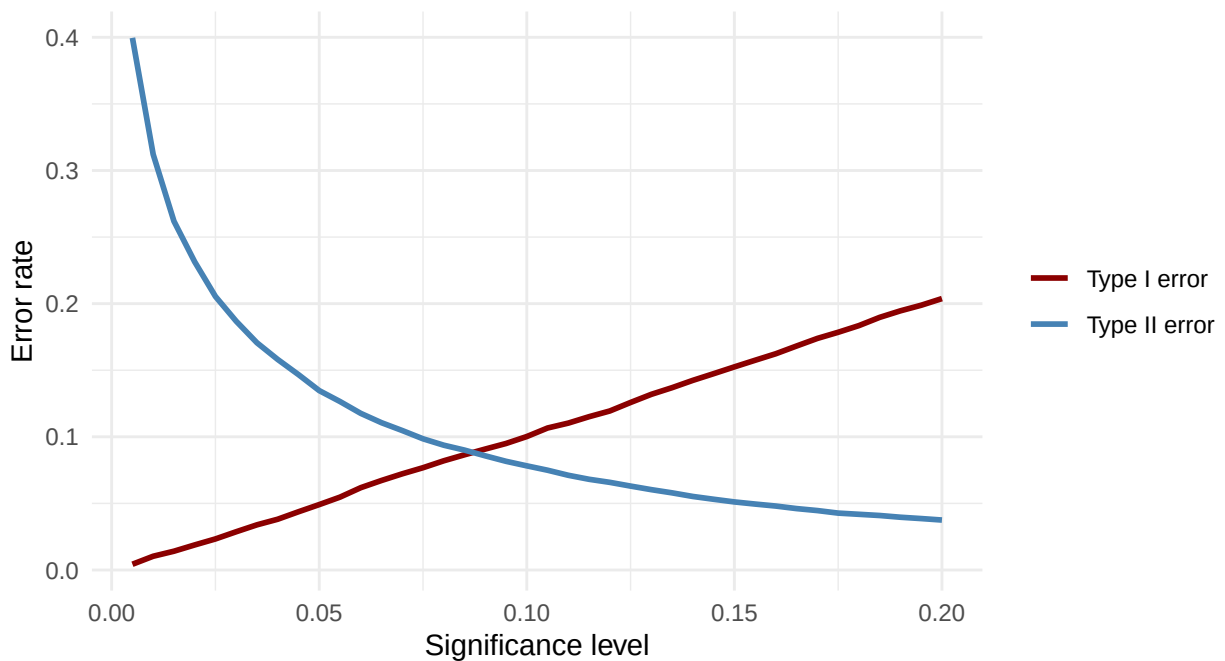
```
alpha_grid <- seq(0.005, 0.20, by = 0.005)
grid_table <- do.call(rbind, lapply(alpha_grid, error_rates))

grid_long <- data.frame(
  significance_level = rep(grid_table$significance_level, 2),
  rate = c(grid_table$type1_error, grid_table$type2_error),
  error = rep(c("Type I error", "Type II error"), each = nrow(grid_table))
)

ggplot(grid_long, aes(x = significance_level, y = rate, color = error)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("Type I error" = "darkred", "Type II error" = "steelblue")) +
  labs(
    title = "Type I and Type II error rates across significance levels",
    subtitle = "Null of 1.5 is true, null of 1.3 is false",
    x = "Significance level",
    y = "Error rate",
    color = NULL
  ) +
  theme_minimal()
```

Type I and Type II error rates across significance levels

Null of 1.5 is true, null of 1.3 is false



Summary and questions to think about

We reused the population from Lab 2, drew 10,000 samples of 10,000 observations, and stored the slope and its standard error from every regression. Testing a true null showed that the test errs about as often as the significance level says it will, which is the Type I error rate. Testing a false null showed that the test misses the truth some of the time, which is the Type II error rate, and that power is one minus that rate. Finally, changing the significance level showed that a stricter test lowers the Type I error rate but raises the Type II error rate, so the two errors cannot be reduced together without more data or less noise.

Before you submit this lab, work through the following questions.

1. In the significance level comparison, explain in your own words why the Type I error rate equals the significance level while the Type II error rate does not.
2. Change the false null from 1.3 to 1.0. What happens to the Type II error rate, and why.
3. Change `n_sample` to 500 and rerun everything. What happens to the Type I error rate and to the power, and why do the two respond differently.
4. Change `sigma_error` to 40 and rerun. How does a noisier error term affect the standard error, and through it the power of the test.